Lab 6 Report: TinyML Anomaly Detection

EE 446 TinyML - Lab 6

Section 1: Edge Impulse Results

Public Edge Impulse project link: [paste the project URL here]

The submission guidelines require the public Edge Impulse project link inside this PDF. The URL was not visible as plain text in the chat, so paste it into this line before uploading to GitHub/Canvas.

Part 1 checklist to complete from the Edge Impulse project:

- Raw inputs used in the fan anomaly detection project.

- NN classifier feature type, number of input features, and at least five actual feature names.

- NN classifier outputs/classes.

- Anomaly detector feature type, number of features, and feature names.

- Screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

- Short interpretation of the observed results and when classifier output may or may not be reliable.

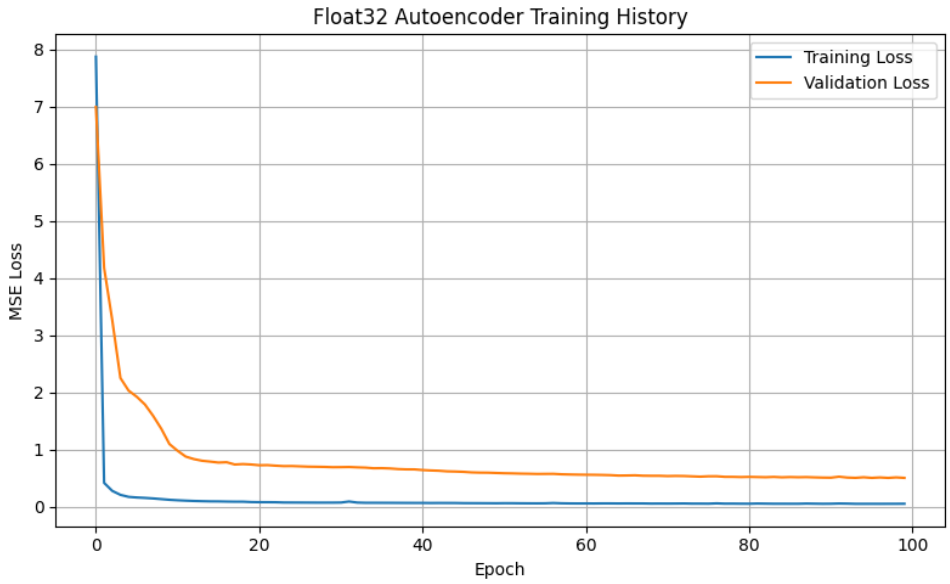
Section 2: Autoencoder Model Results

Dataset preparation

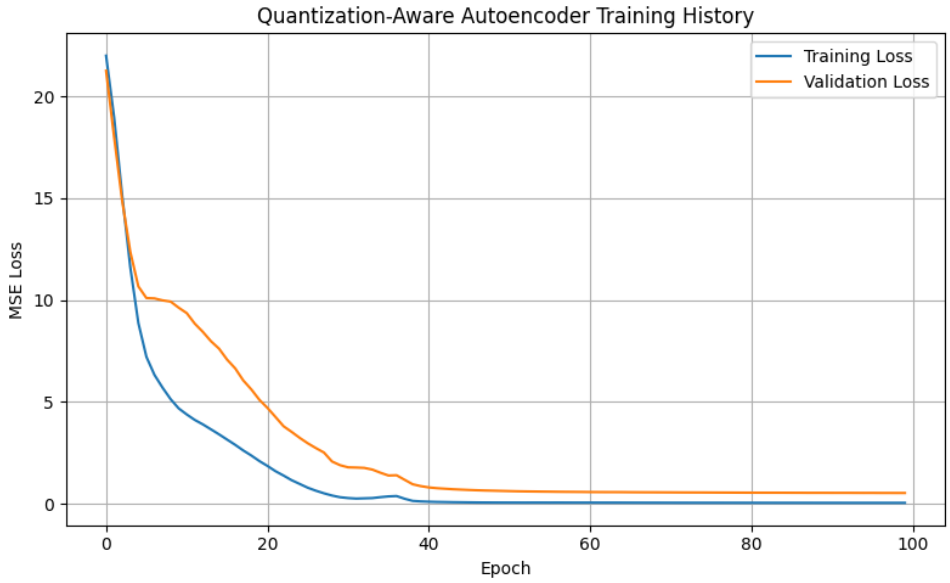
```
X_train shape: (21350, 300) X_test shape: (8479, 300) y_train shape: (21350,) y_test shape: (8479,) Training activity distribution: [2051 2051 2051 2051 2051 1442 1370 1513 2051 2051 2051 617] Testing activity distribution: [823 823 823 823 823 562 531 593 823 823 823 209] Training binary distribution [normal, anomaly]: [10478 10872] Testing binary distribution [normal, anomaly]: [4155 4324]
```

Training and Validation Loss Curves

Float32 autoencoder loss

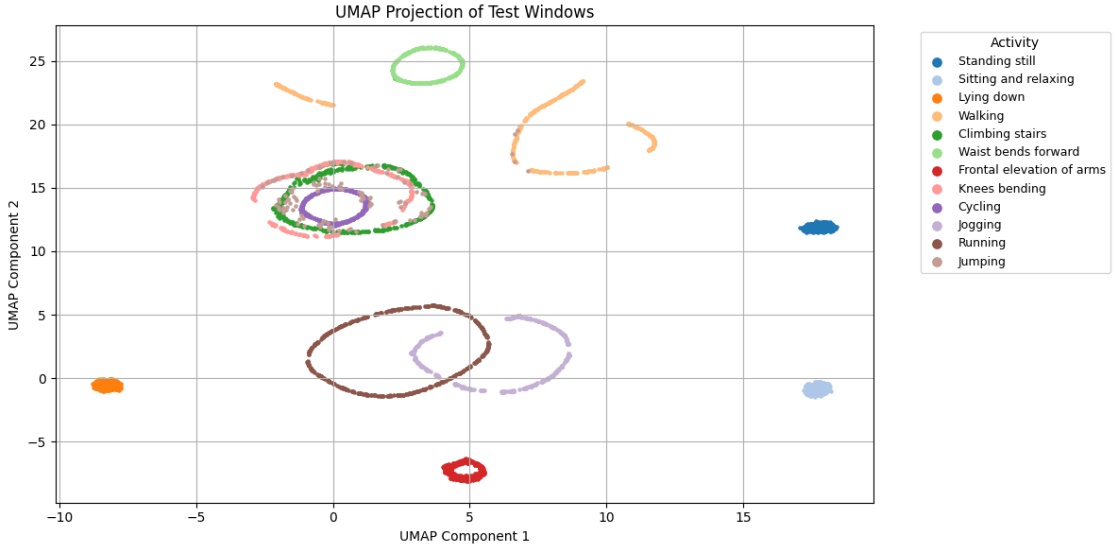


Quantization-aware autoencoder loss

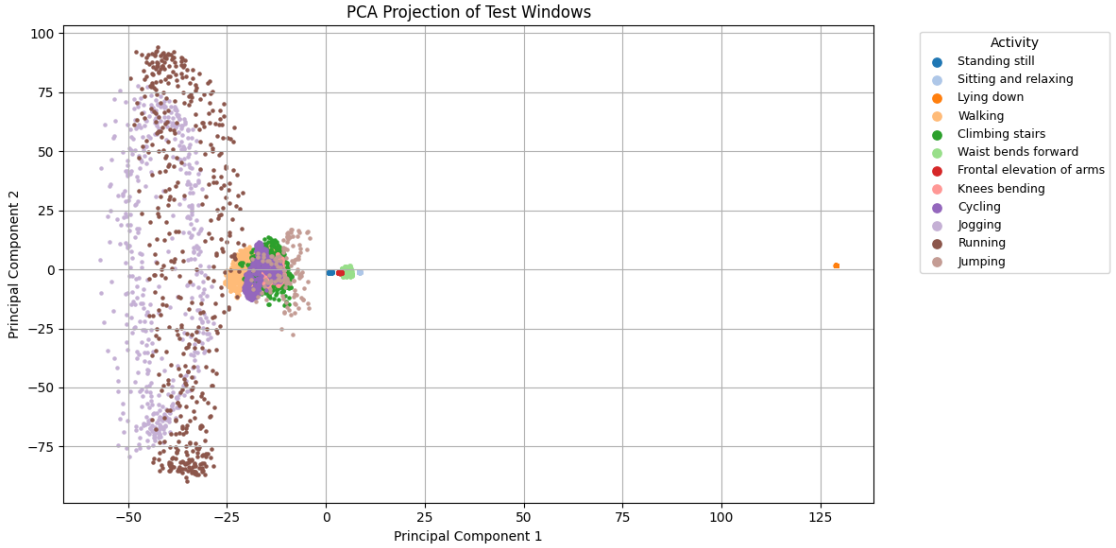


Dimensionality Reduction Visualizations

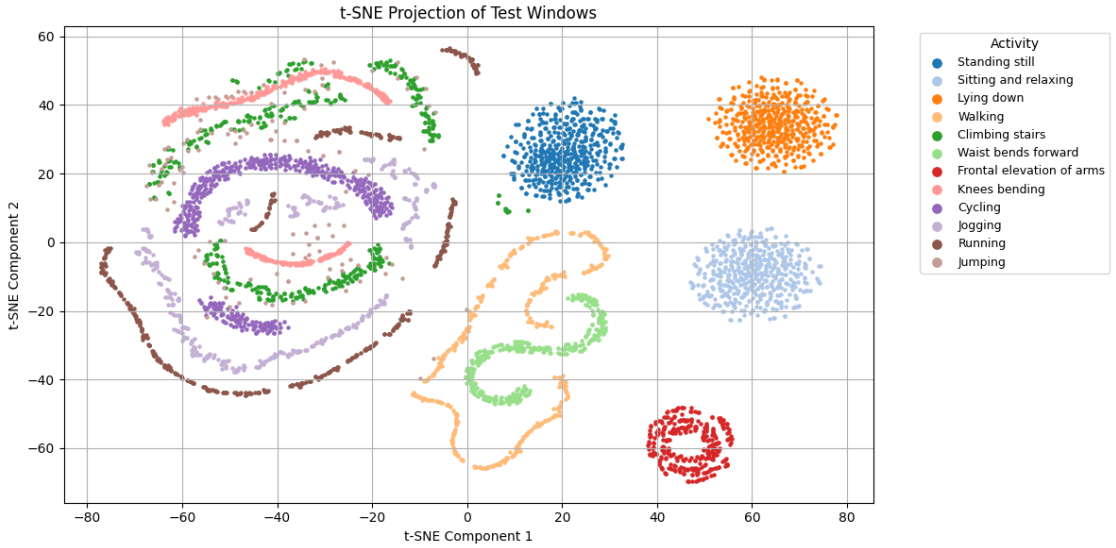
UMAP projection



PCA projection

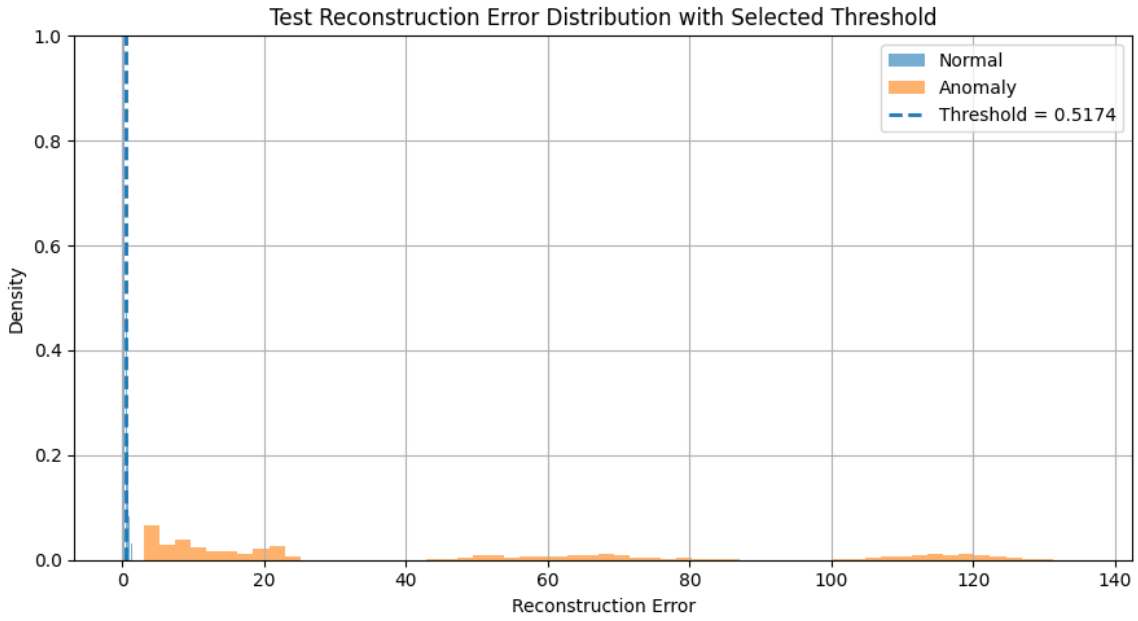


t-SNE projection



Reconstruction Error and Selected Threshold

Selected anomaly threshold: 0.517380. The threshold was chosen as the 95th percentile of training-normal reconstruction errors. This keeps most normal windows below threshold while anomaly windows have substantially larger reconstruction errors.

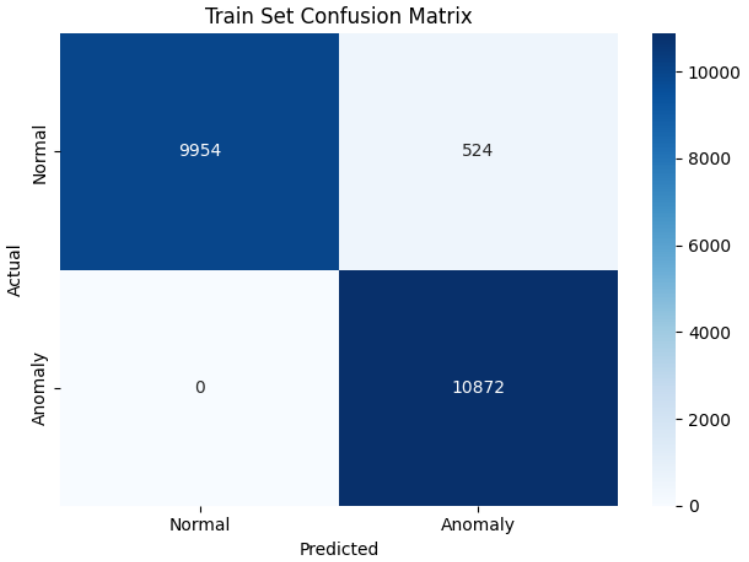


Classification report

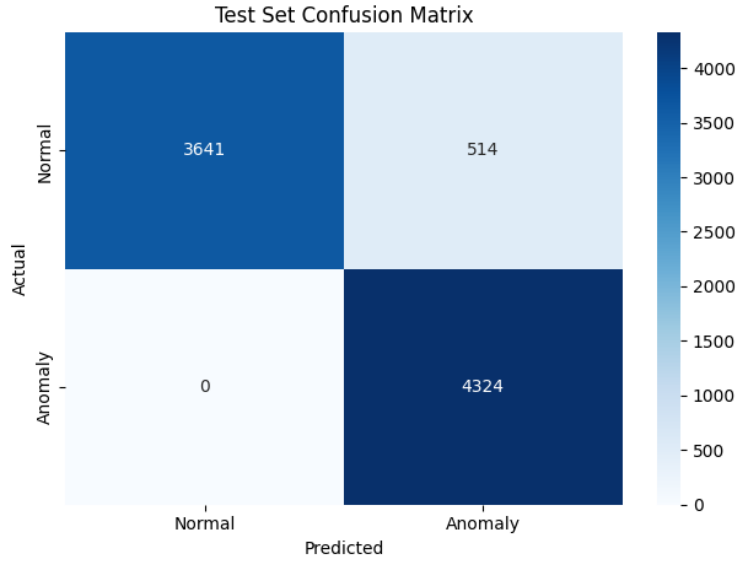
```
Train normal mean=0.101411, stderr=0.001721, min=0.007937, max=1.081591 Train anomaly mean=43.264015, stderr=0.380491,
min=2.377850, max=135.652527 Test normal mean=0.140311, stderr=0.003999, min=0.009032, max=1.392442 Test anomaly
mean=43.636127, stderr=0.635301, min=2.978634, max=135.663330 Selected threshold: 0.517380 Train set evaluation precision
recall f1-score support Normal (0) 1.00 0.95 0.97 10478 Anomaly (1) 0.95 1.00 0.98 10872 accuracy 0.98 21350 macro avg 0.98
0.97 0.98 21350 weighted avg 0.98 0.98 0.98 21350 Test set evaluation precision recall f1-score support Normal (0) 1.00 0.88
0.93 4155 Anomaly (1) 0.89 1.00 0.94 4324 accuracy 0.94 8479 macro avg 0.95 0.94 0.94 8479 weighted avg 0.95 0.94 0.94 8479
```

Confusion Matrices

Train confusion matrix



Test confusion matrix



Int8 Model and Arduino Deployment Evidence

```
INFO:tensorflow:Assets written to: C:\Users\otaku\AppData\Local\Temp\tmptn_ugpgo\assets INFO:tensorflow:Assets written to:
C:\Users\otaku\AppData\Local\Temp\tmptn_ugpgo\assets
C:\Users\otaku\AppData\Roaming\Python\Python39\site-packages\tensorflow\lite\python\convert.py:947: UserWarning: Statistics
for quantized inputs were expected, but not specified; continuing anyway. warnings.warn( Saved autoencoder_int8.tflite Model
size: 24.46 KB Input quantization: (0.08135765045881271, 18) Output quantization: (0.07594722509384155, 8) Train set
evaluation: int8 model precision recall f1-score support Normal (0) 1.00 0.95 0.97 10478 Anomaly (1) 0.95 1.00 0.98 10872
accuracy 0.98 21350 macro avg 0.98 0.97 0.98 21350 weighted avg 0.98 0.98 0.98 21350 Test set evaluation: int8 model precision
recall f1-score support Normal (0) 1.00 0.88 0.93 4155 Anomaly (1) 0.89 1.00 0.94 4324 accuracy 0.94 8479 macro avg 0.95 0.94
0.94 8479 weighted avg 0.95 0.94 0.94 8479 Estimated tensor memory: 21.90 KB INFO:tensorflow:Assets written to:
C:\Users\otaku\AppData\Local\Temp\tmpjokxxh_x\assets INFO:tensorflow:Assets written to:
C:\Users\otaku\AppData\Local\Temp\tmpjokxxh_x\assets TFLite model size comparison Float32 TFLite : 82.87 KB Full int8 TFLite :
24.46 KB
```

Arduino serial monitor output - normal activity screenshot transcript

```
Reconstruction error: 0.408825 Result: Normal activity Reconstruction error: 0.416903 Result: Normal activity Reconstruction
error: 0.416909 Result: Normal activity Reconstruction error: 0.416804 Result: Normal activity Reconstruction error: 0.416701
Result: Normal activity Reconstruction error: 0.416571 Result: Normal activity Reconstruction error: 0.416438 Result: Normal
activity Reconstruction error: 0.416367 Result: Normal activity
```

Arduino serial monitor output - anomaly screenshot transcript

```
Reconstruction error: 1.942497 Result: Anomaly detected Reconstruction error: 2.058063 Result: Anomaly detected Reconstruction
error: 2.453112 Result: Anomaly detected Reconstruction error: 3.203115 Result: Anomaly detected Reconstruction error:
2.918711 Result: Anomaly detected Reconstruction error: 2.545860 Result: Anomaly detected Reconstruction error: 2.569746
Result: Anomaly detected Reconstruction error: 2.711610 Result: Anomaly detected Reconstruction error: 2.853038 Result:
Anomaly detected
```

Interpretation: values around 0.416 are below the selected threshold of 0.517380 and are classified as Normal activity. Values from about 1.94 to 3.20 are above threshold and are classified as Anomaly detected, matching the intended behavior of the deployed autoencoder.

Section 3: Discussion Questions

What threshold did you choose for anomaly detection, and why?

I chose 0.517380, the 95th percentile of the training-normal reconstruction errors. This threshold preserves most normal windows while still separating anomaly windows, whose reconstruction errors were much larger in both train and test splits.

How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder is trained only on normal activities, so it learns to reconstruct those input patterns well. Activities outside that learned normal distribution reconstruct poorly and therefore produce higher mean squared reconstruction error.

What information from the Python notebook must be preserved when deploying the model to Arduino?

The Arduino deployment must preserve the trained int8 model, selected threshold, 100-sample by 3-axis input window shape, flattening order, accelerometer-axis assumptions, sampling behavior, and int8 quantization behavior used by TensorFlow Lite Micro.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The model expects features in the exact same order and scale it saw during training. If the Arduino sketch changes sensor axes, window length, flattening order, or sampling behavior, the reconstruction error threshold becomes unreliable.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The method is sensitive to sensor placement, orientation, subject differences, threshold choice, and real-world motion not represented in training. It also reports whether a window looks abnormal rather than identifying the exact activity class, while memory and compute constraints limit model complexity.